

AUTOMODE.

Think Bigger.

A tech report on a way of working that pays off when you set it up properly — and fails expensively if you mistake it for "using an AI tool."

ENGINEER → ENGINEER · WITH CODE, WITHOUT HYPE VOCABULARY

The shift of the past two years isn't that models can suddenly write code — that's the surface. The shift underneath is the role a single human plays in the process. **From typing to commissioning. From editor to dispatcher.** Andreas calls this *Automode* internally, and the name fits: the machine runs autonomously, the human sets goals, reads outputs, intervenes at the right places.

This text explains how Automode works mechanically, which five building blocks you need, how the loop runs — and where it breaks reproducibly. It's written for people who build software, not for people who talk about it.

One distinction up front — because *using AI* is not one thing but three, and they differ in kind, not degree:

Level 1 — Autocomplete. The model sits in your IDE and finishes your lines as you type. You stay in full control over every character. The tool got faster, the activity is the same as ever.

Level 2 — DwT mode (*Developer with Tool*). You sit in front of a CLI agent like Claude and hand it a concrete task — "*implement this function,*" "*fix this test,*" "*add logging*" — and let it do that on its own. You're still the developer, but you're now delegating whole work packages instead of single lines. At the end you look at it, accept or correct.

Level 3 — Automode. You no longer sit *in front of* an agent, you sit *above* a fleet. You guide an orchestrating Claude that itself spawns a team of sub-agents — and each of those sub-agents can spawn its own team again. The commissioned team becomes a **team-of-teams**, a tree of processes with a single human at its root. That's the level this text is about.

All three are legitimate ways of working, and they solve different problems. The mistake this text wants to correct is the assumption that Level 3 is just a particularly enthusiastic Level 2. It isn't. It differs in *structure*, not in scope — different input, different output, different way of scaling, different way of failing.

§ 01 · THE SHIFT

Why writing code stops being a competitive advantage

I made the economic case in a previous text some time ago. For brevity I'll quote myself instead of re-deriving it here:

Good developers don't become obsolete — they get exposed. Anyone who could only implement, without understanding, will lose relevance.

— AI SAAS WARS: WHEN CODE BECOMES CHEAP · CHRISTIAN.BUERCKERT.EU/KI-SAAS-WARS-WENN-CODE-BILLIG-WIRD-TRENNT-SICH-INTELLECTUAL-PROPERTY-VON-COMMODITY-SOFTWARE

Short version: code is on its way to being a commodity. The moat “we can build that” is melting. What stays is what stays hard even when ten good people show up with an AI setup — domain knowledge, data flow, regulatory safety, operational stability. I called that *Intellectual Property* in that piece. For us as engineers, the consequence is concrete: the work shifts from implementation to translation.

An example from my own practice, so the thesis doesn't stay abstract: I tested whether Claude could invent my patent. It couldn't. After I had explained it to it, it could implement it cleanly — the way any second-semester CS student would manage. The value isn't in the implementation. It's in having the right idea at the right time. And even that isn't enough: you need someone who wants the result, someone willing to pay for it, the implementation, the people who use it, integration into existing systems, an organisation around it. **All of that is IP. Code is not anymore.**

And exactly here, two self-images in our trade collide. The old one: I write code, I measure my contribution in commits and pull requests. The new one: I formulate a problem so cleanly that a pipeline of processes can solve it — and I build that pipeline. The output looks similar; the activity does not.

LEVEL 1 · AUTOCOMPLETE

I type, the model completes

Unit: the line

Context: in my head

Review: while typing

Scaling: type faster

Moat: “I type faster”

LEVEL 2 · DWT MODE

I commission tasks, the agent runs them

Unit: the task

Context: in the chat thread

Review: after each task

Scaling: parallel tasks

Moat: “I know what I want to build”

LEVEL 3 · AUTOMODE

I commission teams that commission teams

Unit: the sprint, the artefact

Context: on the shared blackboard

Review: role discipline + reviewer agent

Scaling: recursive, team-of-teams

Moat: “I can order it”

The sentence the rest of the mechanics revolves around is deliberately simple:

Don't be a developer with AI as a tool.

Be someone who commissions a team that itself commissions teams.

Simple, because it's a thinking operation, not a tool decision. The same model on all three levels produces completely different results. The difference doesn't sit in the IDE, it sits between the ears of whoever sits in front of the IDE — and in how deep the tree is that they unfold behind it.

§ 02 · THE ANATOMY

Five building blocks — and what happens when one is missing

Automode is not a vague principle but a concrete pattern made of five building blocks that presuppose each other. Take one out and the rest collapses, and you're back in DWT mode — just with more expensive tokens. The formal side in half a line:

```

// Automode, in five variables
Automode := (Charter, Roles, Blackboard, Tick, Log)

// The loop
while (bb.hasOpenIssues()) {
    plan    = orchestrator.plan(bb.listIssues())    // reads board, picks scope
    human   = checkpoint(plan)                     // the only manual step
    result  = spawn(roles, plan.scope).execute()    // parallel, isolated
    review  = reviewer.verify(result, charter)      // checked against the rules
    log.commit(review)                             // artefact, git commit
    bb.closeIssues(review.done)                     // board updates itself
    bb.createIssues(review.findings)                // new findings as tickets
}

```

Everything that follows is the explanation of these six lines. None of the building blocks is optional. The most common failure modes — we'll get to those — can almost always be traced back to one line that someone left out.

I **The Charter**

A single document that your team reads before doing anything: what is the product, which stack do we use, which conventions hold, who has which role, how do we commit. No philosophy — an operating manual.

Every organisation that works has one of these. Most developers never wrote it down because it was “obvious.” The moment you delegate, nothing is “obvious” anymore.

II **The Roles**

Not tasks, but roles. A bug-fixer fixes bugs. A DevOps agent thinks about pipelines and containers. A reviewer checks whether the others built nonsense. Whoever delegates tasks has to re-contextualise for every single task. Whoever establishes roles contextualises once and reaps for a long time.

The role matrix is the most expensive part of the upfront work — and the one nobody skips without regretting it later.

III **The Blackboard**

State doesn't live in agents. Agents are stateless. State lives on a board that everyone can read and write — tickets in an issue tracker, markdown files in the repo, a backlog list. The point: when the next sprint starts, what the previous one left behind is sitting there.

That's the blackboard pattern from 1980s AI research, only that the knowledge sources are language models now.

IV **The Tick**

Work runs in fixed intervals. Not “when done,” but “every X minutes / per sprint / per ticket.” The tick forces closure. Without a tick, an agent works hours on something that should have been stopped after ten minutes.

Short ticks aren't there to make you faster — they're there to give you more decision moments.

V **The Log**

Every tick ends in a verifiable artefact: a commit, a closed issue, a PR, a deployment log. If at the end of a sprint nothing sits in any log, the sprint produced nothing. The rule is harder than it sounds.

Git is not just version control when you work this way. Git is the black box of your fleet.

Three phases that repeat endlessly

The five elements join into a cycle that you remember easily, because it resembles Scrum — and at the same time is something completely different. Scrum for humans wants to synchronise communication; this loop wants to synchronise delegation.



Planning — the only point at which you are indispensable

In phase one, *one* orchestrating agent — or you yourself — reads the blackboard and decides what happens this tick. Which tickets, which roles, which priority. Then there is exactly **one human checkpoint**: read it over, sign off or correct. This checkpoint is the reason the machine doesn't spend four hours working on something you no longer want.

Execution — where the illusion of speed comes from

Several roles run at the same time, ideally in isolated work copies (branches, worktrees, containers). Each role has exactly one mandate and a defined set of files it's allowed to touch. Parallelism here works not despite but *because of* the strict role boundaries — two agents writing in the same code produce merge conflicts, not progress.

Review — the step many solo developers skip, even though it's the most important one

Every tick ends with integration. All results get merged, a build pipeline runs through, a review agent reads the overall result against the charter, metrics get updated, a commit is created with the tick number in the title. If the tick produces nothing committable, exactly that is written openly into the log and the cause is added to the next tick. Never sugarcoat anything, never hide half work.

An archetype set that maps onto almost any product

The roles below are not an invention; they exist in every serious software organisation. The trick is not to treat them as job descriptions but as **prompt profiles**. Every role is a well-formed mandate that comes with a clear scope, a bounded set of files, and a completion criterion.

ROLE 01

Feature Builder

Implements the next backlog item. Reads acceptance criteria, implements, writes tests.

ROLE 02

Bug Hunter

Works through the bug list. Reproduces, fixes, verifies. Files new findings.

ROLE 03

Reliability

Makes sure the system builds, starts, stays up. Endpoints, pipelines, logs.

ROLE 04

Refactorer

Improves code quality against a fixed standard. No features, no breaking changes.

ROLE 05

QA / E2E

Runs end-to-end tests, generates screenshots, files UX findings as new tickets.

ROLE 06

Reviewer

Reads the tick's overall output against charter and architecture rules. May reject.

ROLE 07

DevOps

Takes care of containers, pipelines, secrets, deployment. No application logic.

ROLE 08

Designer

Before UI-heavy ticks: analyses screenshots, writes redesign specs others implement.

ROLE 09

Product Owner

Triages issues, prioritises, closes duplicate tickets, formulates milestones.

ROLE 10

Researcher

Reads external documentation, gathers facts, writes decision memos. Writes no code.

The number is not sacred — and above all not stable. Early in the project, three architects often matter more than ten implementers. As soon as the structures stand, the ratio flips: twenty bug-hunters run in parallel while the architects fade into the background. **The role lineup wanders with the project state, not the other way around.**

The advantage that really carries this: **these roles are not people**. If you no longer need a role, you're not sending anyone home — nobody ever showed up. If a role that used to fit no longer suits, you rewrite its prompt. No HR conversation, no goodbye, no career setback, no family that depends on the role going away. Roles are abstract descriptions that a model brings to life for the duration of an assignment; underneath, everything lives in the LLM. That makes restructuring trivial — and allows a flexibility that human teams will never have, for very good, very human reasons.

What counts is therefore not the fixed number but the principle: **every role is sharply bounded, every role has its files, every role delivers an artefact**. Which roles are filled at any given moment is something you decide per tick — and you adapt it to what is currently needed, with no regard for continuities that don't exist.

Team of Teams — the actual lever

What qualitatively distinguishes Automode from the two lower levels is this property: **every role can itself be an orchestrator**. The bug hunter you commission doesn't have to be a single process working through ten bugs sequentially. It can spawn a sub-team — ten parallel bug-fixers, each on one bug, plus a review agent that bundles their work before it's handed back up. A role becomes its own loop with its own charter, its own blackboard, its own log.

```
// Recursive structure – simplified
```

```
You
```

```
└─ Orchestrator
```

```
    │ Feature Builder
```

```
    │ └─ spawns: Schema-Designer, API-Builder, UI-Builder
```

```
    │ Bug Hunter
```

```
    │ └─ spawns: 10× Bug-Fixer parallel, 1× Review
```

```
    │ QA
```

```
    │ └─ spawns: E2E-Runner, Screenshot-Differ, UX-Scout
```

```
    └─ Reviewer
```

```
        └─ spawns: Security, Perf, Arch-Check
```

This tree has neither a fixed depth nor a fixed width. A trivial ticket stays a single execution. A multi-layered feature expands into a three-storey team-of-teams that runs through the code in parallel and at the end returns one single result. The human at the root only ever sees the planning step and the review — the entire tree in between is, for them, one single mandate, not fifty.

This is the point at which scaling really kicks in. Level 1 (Autocomplete) scales with your typing speed. Level 2 (DwT) scales with the number of parallel tasks you can sensibly hold in your head — realistically two, three. Level 3 (Automode) scales with the *depth of the tree* you can delegate, and that's not technically bounded. It is bounded by the quality of your charter and your ticket formulations — and with that, we're back at the start.

And a Level 4?

Probably yes. It's likely not "the charter writes itself" or anything that esoteric, but simply **one level thicker**. If Level 3 is the team-of-teams tree, Level 4 is the **organisations-of-teams-of-teams** — multiple trees side by side, talking to each other.

At Level 3 you steer a single team-of-teams tree: one product, one codebase, one charter. At Level 4, the same human steers **several such trees at once**, and the trees communicate among themselves. The marketing Automode talks to the product Automode talks to the support Automode — Claudes-talk-to-Claudes, over protocols like MCP or whatever ends up establishing itself for it. Every tree has its own charter, its own loop, its own roles. They negotiate interfaces with each other, tickets travel from tree to tree, reviews run across organisational boundaries.

The human at the top is then no longer the product manager of one team but the chief executive of an organisation in which every function runs its own Automode. They set direction, goals, allocations — no more tickets, no more roles, no more charters. The trees do all of that themselves.

This isn't wild speculation. The building blocks exist individually today: Automode trees run in individual products, MCP-like protocols connect agent to agent, companies have multiple functions each with their own process. What's missing is the clean wiring across all of it — and whoever builds it first will name it.

The reduction curve stays consistent:

- **Level 1** reduces typing time.
- **Level 2** reduces task time.
- **Level 3** reduces management time *of one team*.
- **Level 4** reduces coordination time *between teams*.

What remains as IP shifts up another layer with that. At Level 3 your IP is the translation layer: charter, tickets, checkpoints. At Level 4 your IP is **direction and allocation** — where does the energy of the organisation flow, which trees get which resources, which goals talk to each other. The last residue that can't be machine-produced.

§ 05 · THE BLACKBOARD

How the team knows what to do today

State not in heads, not in chat threads, not in "I told you that...". State on a board everyone can read, that's versioned, that's checkable. There are two practical build-out levels, depending on how serious you are about it.

Level 1 — markdown in the repo (the entry point)

Minimal setup, zero infrastructure, works from day one. Anyone who has never worked in Automode should start here — a few files in the repo, git as the log, done:

```
# Repository root
CHARTER.md      → Product vision, stack, conventions, roles
BACKLOG.md      → What is to be built, by priority
BUGS.md         → Active defects, by severity
ARCHIV.md       → What has already been done (memory)
reviews/        → One report per tick
adr/            → Architecture decisions, in dialogue
```

The advantage: everything in git, mergeable, greppable, human-readable. The disadvantage: beyond a certain volume of tickets, the markdown format becomes the bottleneck. Prioritisation in a table, dependencies in lists, burndown by hand — at some point you notice you’re rebuilding a real tracker.

Level 2 — blackboard as MCP service (from medium complexity onward)

At the point where the markdown solution starts groaning, a real issue tracker that the agents speak to via the **Model Context Protocol** pays off. Concretely, this works with a small service such as `mcp-blackboard` — a local Node server with SQLite, REST API, web UI, and a handful of MCP tools that every agent calls natively:

```
// Instead of "grep BACKLOG.md", the agent calls directly:
bb_list_issues(project, status="open", label="high-priority")
bb_get_issue("APP-142")
bb_update_issue("APP-142", status="in_progress", assignee="dev-agent-2")
bb_add_comment("APP-142", "Fixed in commit a1b2c3d")
bb_close_issue("APP-142")

// The planning phase becomes trivial:
bb_get_board(project)           // kanban state
bb_get_burndown(milestone)      // progress in the sprint
bb_list_acceptance_criteria(id) // what counts as "done"?

// The review phase produces the artefact automatically:
bb_create_sprint_review(sprint, metrics, timeline_html)
```

What you gain from this is not cosmetic:

- **Structured queries instead of text search.** “All open high-prio bugs in the current milestone” is a function call, not a regex.

- **Dependencies as a first-class citizen.** Blocker relationships, critical path, “what blocks what” — barely maintainable in markdown, native in a tracker.
- **History for free.** Status changes, comments, assignments are logged automatically. Burndown and velocity fall out as curves, not as manual calculation.
- **Web UI for humans, API for agents.** Same data source, two interfaces — you see in the browser what the machine sees, and vice versa. No syncing needed.
- **Cross-project.** One server, many projects. Automode scales across repo boundaries without a per-project markdown bureaucracy.

Pragmatically: as soon as a sprint has ten or more tickets, the MCP variant is by far the cleaner solution. Agents no longer parse files; they query the board directly, and the human gets a kanban interface in the browser that beats every Jira-Lite.

Two rules apply to both stages, and both are easy to overlook:

First: the blackboard is the agent. If a ticket is sloppily written, the team produces sloppy work — completely independent of the model you use. The quality of your ticket formulations is the upper bound of your output quality. Most of the time you save isn’t saved at programming time; it’s saved upstream, while writing good tickets.

Second: the blackboard doesn’t replace your notes, it replaces your meeting. If you know something the machine needs to know, write it on the blackboard. Not into a chat, not into a commit message, not into your head.

§ 06 · APPLICATION

How you start tomorrow — independent of the product

The pattern is product-neutral. Whether you’re building a SaaS, an internal data pipeline, a mobile app, a library, a documentation portal — the six steps below are the same. Only the role contents and the blackboard vocabulary vary.

01 Write your charter before you do anything else.

A single file. Product name, mission, tech stack, conventions, directory layout, build commands, commit conventions. It must be readable in ten minutes — not ten hours.

02 Declare yourself the PM, not the developer.

This sentence belongs verbatim in the charter: “I am the project manager. I spawn roles. I write no production code.” This is not a pose, it’s a role assignment your agents read and orient themselves against.

03 Define the roles your project needs *today* — the number will wander.

Start with few, often three. Spawn another as soon as an area earns its own specialisation, and shrink back when a phase is over. Every role gets: when it's spawned, which files it touches, what its completion criterion is. One sentence per point is enough.

04 Set up your blackboard.

For the start, three markdown files in the repo are enough — backlog, bugs, archive. As soon as the ticket volume grows, you switch to a real tracker your agents speak to via MCP (e.g. `mcp-blackboard` or a GitLab/GitHub project). Fill it with everything floating around in your head. Your first sprint hour is a writing exercise, not a coding exercise.

05 Establish the tick.

A sprint cadence that fits your tempo: thirty minutes, two hours, a working day. What matters is regularity and a completion criterion. A tick without a commit is a failed tick.

06 Make the first tick deliberately small.

A trivial bug, a clean-up, a documentation update. The first tick exists to test the mechanics — not to produce progress. When the first tick runs through cleanly, the machine is calibrated.

What is striking about this order is how strongly it contradicts how developers normally start. Usually we start by building and document backwards. In Automode, *documenting is building*. What emerges in the first hours isn't overhead — it is the production machine itself. The code then almost falls out as a by-product.

§ 07 · LIMITS

Where the pattern breaks — and has to honestly break

Big design decisions can't be parallelised. When a fundamental UX direction, an information architecture, an API contract has to come into being, that doesn't belong inside a multi-agent tick. There you sit yourself, with a single model, and think it through.

Foundational decisions where you don't yet know what you want are hard to delegate — simply because the mandate stays unclear. An architect agent helps here as a dialogue partner, not as auto-committer: the decision emerges in conversation, the agent then writes it out. That's not a break of the pattern but a role shift — the human is the sparring partner, not the gatekeeper.

External dependencies without testability (third-party APIs, OCR, LLM calls, physical hardware) need several ticks before they run stably, because the failure patterns only show up live. That's not a bug of the method, it's of reality — but you must factor in that those areas converge more slowly.

The quality of your blackboard is the ceiling above everything. Sloppily maintained tickets lead to sloppy work, guaranteed. When the machine constantly misses the target, it's almost never the machine's fault.

This is the list at which many enthusiasts wave you off and say, well then it doesn't "really" work. The opposite is the case. A tool that knows its limits is a usable tool. A tool that's supposed to do everything is a marketing artefact.

§ 08 · AN UNCOMFORTABLE TRUTH

Not every developer will take this step

This section has to be written, even though it's uncomfortable, because the silent alternative — pretending it's purely a matter of learning — is in the end neither honest toward the other person nor toward the reality we're moving in.

Not everyone who writes code today will switch to Automode. Just as not everyone who programmes well becomes a senior. Senior is not "ten years of junior" — it's a different way of seeing the problem. Being a dispatcher is not "ten years of developer" — it's another different way again. Whether someone makes this shift depends on things that **can be learned, but not by everyone**: abstract formulation, system over detail, being able to let go during execution, trust in processes, joy in delegating instead of doing things yourself.

That sounds harsh, and it's nevertheless true. And it's important to say, because the alternative — pushing everyone with motivational slogans into the same role — is the greater cruelty: putting people into positions in which they fail, even though they would be strong elsewhere.

A developer who doesn't want to become a commissioner isn't worth less. They just work on a different layer of the same value chain.

Because — and this is the half that the tech hype likes to leave under the table — **the execution layer doesn't disappear, it just gets denser**. UX polish, detail reviews, alt-text corrections, button spacing, forms that feel right, error messages that help instead of frustrating, migrations that run cleanly, the small things that together make up the *feel* of a product. None of that happens by itself, just because someone up top commissioned it. Exactly this layer stays human, stays craft, stays handwork — and is not trivial, but **exactly what distinguishes a product from a demo**.

The honest map looks more like this:

PATH A · DISPATCHER

Formulates problems instead of solving them

Maintains charter, backlog, roles

Orchestrates several processes in parallel

Thinks in systems, ticks, artefacts

Measured by throughput and correctness of mandates

PATH B · CRAFTSPERSON

Spots the errors machines don't see

Polishes what the machine roughed in

Knows the codebase deeper than any agent ever will

Thinks in pixels, microcopy, edge cases

Measured by quality and feel of the product

Neither of the two paths is the worse one. The damage happens where someone is rated on Path A even though they're working on Path B — or vice versa. An outstanding craftsperson is not a failed dispatcher. A good dispatcher is not better than a good craftsperson; they do something different.

Can you learn the switch from one to the other? Yes. Will everyone manage it? No. Does everyone have to? Even less. What counts is that both roles are clearly defined, both roles are treated with respect, and above all: **that nobody is forced to choose the wrong one just because someone happens to have written a LinkedIn post saying that “everyone has to learn prompting now.”**

This is the most honest political wrapping I can offer: the shift is real, not everyone will go with it, and the reason isn't diligence or value. It lies in the way someone most likes to work. The job of companies and teams is to build *both* paths properly — not to declare one of them mandatory.

§ 09 · SUMMARY

What you take with you tomorrow

First: the actual shift is a **role shift**, not a tool shift. The same models in DwT mode assist while you type; in Automode they produce teams-from-one-person.

Second: the moat that remains is called **translation**. Between business and machine, between vague idea and precise ticket, between domain knowledge and executable mandate. Everything beneath that becomes increasingly commodity.

Third: the concrete pattern — charter, roles, blackboard, tick, log, loop — is no secret knowledge. It's what well-run software organisations have done for decades. What's new is only that the execution layer no longer consists of seven expensive humans but of seven cheap processes. The boss is still you. What you used to delegate to people, you now delegate to a team that doesn't sleep.

And fourth, the core I was driving towards from the original text: when code becomes cheap, IP separates from commodity. **Your IP is the translation layer — the charter you write, the tickets you formulate, the decisions you make at the checkpoints.** That is the part that stays hard, even when ten good people show up with an AI setup. The rest is typing. Typing the machine can do.

Delegation is the key to scale. What you can't hand off sets your ceiling.

— END —
